

ক্রম নির্ণয় করুন

- সময়সীমা: 10 মিনিট
- পরিবেশ: একটি GPU (≈16 GB VRAM), ইন্টারনেট নেই
- সমাধানের আকার: `solution.ipynb` ≤ 1 MB
- স্টোরেজ: 5 GB

સમસ્યા

আপনাকে দুইজন অংশগ্রহণকারী, *Speaker A* এবং *Speaker B*-এর মধ্যে কথ্য ইংরেজি ডায়ালগ দেওয়া হয়েছে। প্রতিটি ডায়ালগ বক্তার পালায় বিভক্ত, যেখানে প্রতিটি পালায় কেবল একজন বক্তার বক্তব্য থাকে। প্রতিটি পালা একটি পৃথক .wav অডিও ফাইল হিসেবে সংরক্ষিত, ফলে একটি সম্পূর্ণ ডায়ালগকে .wav ফাইলের একটি সেট দ্বারা উপস্থাপন করা হয়, প্রতিটি পালার জন্য একটি করে ফাইল।

দুর্ভাগ্যবশত, পালাগুলো এলোমেলোভাবে অদলবদল করা হয়েছে, তাই কথোপকথনটি আর অর্থপূর্ণ নয়। `chunk_{k}.wav` ফাইলনামে, k বলতে এলোমেলো সেটের k -তম chunk বোঝায়, মূল ডায়ালগের k -তম পালা নয়।

!! আপনার কাজ হলো কথোপকথনের মূল কালানুক্রমিক ক্রম পুনর্গঠন করা।



Dataset

প্রতিটি ডায়ালগে n অডিও ফাইল থাকে, যেগুলোর নাম `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav`। `chunk`-গুলো পৃথক পৃথক পালা। ফাইলনামগুলো কেবল এলোমেলো ক্রমের সঙ্গে সম্পর্কিত। মূল কথোপকথনে কোনো `chunk` কোথায় থাকবে, তা এগুলো নির্দেশ করে না। প্রতিটি ডায়ালগে 7-20টি `chunk` থাকে, mono, 44.1 kHz (আপনি resample করতে পারেন)।

prefix.json প্রতিটি ডায়ালগের প্রথম দুইটি `chunk`-এর ফাইলনাম **index** ধারণ করে। এটি ডায়ালগের প্রকৃত শুরু শনাক্ত করে এবং কথোপকথনটি সামনের দিকে বা পেছনের দিকে পড়ার মধ্যকার দ্ব্যর্থতা দূর করে।

উদাহরণস্বরূপ: 11: [7, 12]-এর অর্থ হলো dialogue 11-এর প্রথম ও দ্বিতীয় পালা যথাক্রমে `chunk_7.wav` এবং `chunk_12.wav`।

আপনি যা পাবেন

আপনি একই **format**-এর দুইটি **folder** পাবেন:

Folder	ডায়ালগ	answers.json?	এটি যে কাজে ব্যবহার করবেন
dataset/train/	1,288	✓ অন্তর্ভুক্ত	আপনার মডেল train / fine-tune করতে
dataset/test_public/	100	✓ অন্তর্ভুক্ত	আপনার pipeline চালাতে এবং স্থানীয়ভাবে self-score করতে

গ্রেডিংয়ের সময়, আপনার `dataset/test_public/` folder-টি স্বচ্ছভাবে একটি hidden evaluation set (public leaderboard-এর জন্য `test_leaderboard_a` এবং final leaderboard-এর জন্য `test_leaderboard_b`) দ্বারা প্রতিস্থাপিত হয়—এগুলোর আকার ও format `dataset/test_public/`-এর মতোই, তবে `answers.json` ছাড়া।

সেই data-তে আপনার notebook আবার execute করা হয় এবং এটি যে `answers.json` ফাইল তৈরি করে, সেটি scoring-এর জন্য ব্যবহৃত হয়। held-out test ডায়ালগগুলো `train`-এর একই distribution থেকে আসে, তাই আপনার স্থানীয় `test_public` score একটি নির্ভরযোগ্য পূর্বাভাস।

Directory structure

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

Output

প্রতিটি ডায়ালগের জন্য, এর audio chunk-গুলোর মূল কালানুক্রমিক ক্রম নির্ধারণ করুন। আপনার prediction হবে $\{0, 1, \dots, n-1\}$ -এর একটি permutation P , যেখানে $P[i]$ হলো `chunk_i.wav`-এর predicted কালানুক্রমিক অবস্থান ($0 =$ প্রথম)।

আপনার output ফাইল `answers.json` প্রতিটি dialogue ID-কে তার predicted permutation-এর সঙ্গে যুক্ত করবে:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

উদাহরণ

একটি ডায়ালগে 3টি এলোমেলো chunk `chunk_0`, `chunk_1`, `chunk_2` রয়েছে:

এলোমেলো chunk	কথিত বিষয়বস্তু	প্রকৃত অবস্থান (rank)
chunk_0.wav	"No worries — I'll send you the notes afterwards."	2 (শেষ)
chunk_1.wav	"Hey, are you coming to the three o'clock meeting?"	0 (প্রথম)
chunk_2.wav	"I can't — I've got a dentist appointment then."	1

প্রকৃত ক্রম হলো **chunk_1** → **chunk_2** → **chunk_0**, তাই $P = [2, 0, 1]$, এবং `prefix.json`-এ $[1, 2]$ থাকে।

⚠ P অবশ্যই একটি যথার্থ permutation হতে হবে: দৈর্ঘ্য n , 0-indexed, প্রতিটি মান ঠিক একবার। duplicate, অনুপস্থিত মান বা range-এর বাইরের entry (যেমন 1-indexed) সেই ডায়ালগের জন্য 0 score পাবে; ফাইলে কোনো ডায়ালগ অনুপস্থিত থাকলেও একই ফল হবে। কোনো malformed বা non-JSON ফাইল প্রত্যাখ্যান করা হবে।

Scoring

এই কাজের scoring হলো **জোড়াভিত্তিক ক্রম-নির্ভুলতা (pairwise ordering accuracy)**। এটি প্রতিটি chunk-জোড়া পরীক্ষা করে এবং জিজ্ঞাসা করে: *দুইটির মধ্যে কোনটি আগে আসা উচিত?* আপনার prediction যদি ground truth-এর মতো একই উত্তর দেয়, তবে একটি জোড়া সঠিক। n টি chunk-সহ একটি ডায়ালগে

$$M = n(n - 1)/2$$

টি জোড়া থাকে; I দ্বারা inversion-এর সংখ্যা—ground truth থেকে ভিন্নভাবে ক্রমবদ্ধ জোড়ার সংখ্যা—বোঝানো হোক:

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

i চূড়ান্ত score হলো split-এর সব ডায়ালগের per-dialogue score-এর গড়।

অনুমোদিত মডেল

এই কাজটি সমাধান করতে training এবং evaluation উভয় সময়েই আপনি কেবল নিচের pre-trained মডেলগুলো ব্যবহার করতে পারবেন। এই সব মডেল ইতোমধ্যে download করা এবং environment-এ উপলভ্য। baseline notebook solution.ipynb-এ এগুলো কীভাবে ব্যবহার করতে হয় তার উদাহরণ দেখতে পারেন। অনুগ্রহ করে লক্ষ করুন, আপনি অন্য কোনো মডেল ব্যবহার করতে পারবেন না এবং আপনার program-এর internet access নেই।

- **Speech representation: wav2vec 2.0। Whisper encoder**-কেও feature extractor হিসেবে ব্যবহার করা যেতে পারে। [wav2vec মডেল কার্ড](#)
- **Automatic speech recognition (ASR): OpenAI Whisper** (যেকোনো size)। [Whisper মডেল কার্ড](#)
- **Language model: Qwen2.5-0.5B**, যা zero-shot হিসেবে অথবা প্রদত্ত train split-এ fine-tune করে ব্যবহার করা যেতে পারে। [Qwen মডেল কার্ড](#) লক্ষ করুন, grade করার সময় আপনার করা যেকোনো training বা fine-tuning এবং evaluation set-এ inference—উভয়ই 10-মিনিটের সীমার মধ্যে সম্পন্ন হতে হবে।

যেভাবে submit করবেন

- `solution.ipynb` খুলুন এবং সব cell চালান। নিশ্চিত করুন যে এটি working directory-তে `answers.json` লিখছে, যেখানে `dataset/test_public/-`এর প্রতিটি ডায়ালগের জন্য একটি permutation রয়েছে (100টি ডায়ালগ)। grade করার সময় notebook-টি hidden test set-এ পুনরায় চালানো হয় এবং সেখানে তৈরি হওয়া `answers.json` score করা হয়।
- চাইলে সমাধানটি উন্নত করুন—অথবা করবেন না; baseline-টিই pipeline validate করে।
- JupyterLab-এর বাম sidebar-এ Git tab খুলুন।
- `solution.ipynb` **Stage** করুন (এর পাশের + icon)।
- একটি commit message লিখে **Commit**-এ click করুন।
- push করতে cloud-with-up-arrow-এ click করুন।
- এই Contest page-এ ফিরে এসে **Submit**-এ click করুন।

ঠিক একটি ফাইল submit করুন, যার নাম `solution.ipynb`।